

ChefVision AI

Backend Infrastructure, Cloud Deployment, and Ingredient Normalization Service

Design Specification | Version 1.0

Girith Choudhary

girith.choudhary@sjsu.edu

SJSU CMPE 295A | Advisor: Dr. Wencen Wu | April 2026

Version History

Version	Date	Author	Changes
1.0	April 9, 2026	Girith Choudhary	Initial submission covering backend infrastructure, cloud deployment, and ingredient normalization service

Introduction

This document is the individual design specification for Girith Choudhary's contributions to ChefVision AI, a photo-first restaurant inventory management platform. ChefVision AI allows restaurant owners to upload invoice photos, extract line items automatically, normalize ingredient names, plan events, and generate reorder lists.

This specification covers three areas. The first is the backend API and database layer, which handles authentication, invoice processing, inventory management, and event planning logic. The second is the cloud deployment infrastructure, which defines how all services are containerized, deployed to AWS, and operated in production. The third is the ingredient normalization service, which is the ML component responsible for resolving raw vendor-supplied ingredient names to a canonical ingredient catalog.

The project workbook documents these areas at a summary level. This document provides the detailed design, data structures, API contracts with payload examples, deployment configuration, debug and logging approach, and test plan required to build and verify each area.

References

1. ChefVision AI Project Workbook, Choudhary, Pham, Chhabra, April 4, 2026.

2. Mudgal et al. "Deep Learning for Entity Matching: A Design Space Exploration." SIGMOD 2018. <https://dl.acm.org/doi/10.1145/3183713.3196926>.
3. Reimers and Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." EMNLP 2019. <https://arxiv.org/abs/1908.10084>.
4. RecipeNLG dataset. paultimothymooney/recipeNlg on Kaggle. <https://www.kaggle.com/datasets/paultimothymooney/recipeNlg>.
5. pgvector: open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>.
6. AWS ECS Fargate documentation. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html.
7. BullMQ documentation. <https://docs.bullmq.io>.

Requirements

Backend API

- The API must support JWT-based authentication with two roles: restaurant owner and admin.
- The API must accept invoice uploads as image files (JPEG, PNG) and PDF files.
- Invoice processing must be asynchronous. The upload endpoint must return a job ID immediately.
- The API must expose endpoints for inventory viewing, inventory export as Excel, event creation, and reorder list generation.
- Failed processing jobs must retry up to three times with exponential backoff before marking the invoice as errored.

Cloud Deployment

- All services must run as Docker containers and be deployable to AWS ECS Fargate.
- No secrets may be stored in the repository or baked into Docker images.
- The CI/CD pipeline must run the full test suite on every pull request before any deployment.
- The production environment must have multi-AZ database deployment and automated daily backups.
- API uptime in staging must be 99% or above.

Ingredient Normalization Service

- Given a raw ingredient string, the service must return a ranked list of candidate canonical ingredients with confidence scores between 0 and 1.
- Matches with confidence 0.88 or above must be committed automatically without human review.
- Matches with confidence between 0.55 and 0.87 must be flagged for user confirmation.
- Strings with confidence below 0.55 must create a new inactive canonical placeholder for admin review.
- Every confirmed user override must be written back as a new alias so the same string auto-matches in the future.

- Single-item normalization must complete within 200ms at p95. A 50-item batch must complete within 3 seconds.
- Pairwise F1 on the labeled evaluation set must reach 0.85 or above.

Functional Overview

Backend API and Database

The backend is split into two services. The main API server (Node.js + Express) handles authentication, business logic, and database access. The ML service (Python + FastAPI) handles ingredient normalization. Both run as separate Docker containers on a shared internal network.

Layer	Technology	Purpose
Main API	Node.js + Express	REST routes, auth, and orchestration between services
ML Service	Python + FastAPI	Inference for normalization and confidence scoring
Database	PostgreSQL 16 + pgvector	Relational storage and vector similarity search
Job Queue	BullMQ on Redis	Async invoice processing with retry logic
Object Storage	AWS S3	Uploaded invoice images and PDF files
Cache	Redis	Catalog embeddings and session tokens

The main REST API endpoints are listed below.

Method	Endpoint	Description
POST	/auth/login	Returns JWT token for restaurant owner or admin
POST	/invoices/upload	Uploads invoice image, enqueues extraction job, returns job ID
GET	/invoices/:id	Returns invoice with extraction status and line items
POST	/invoices/:id/confirm	User confirms or corrects extracted line items
GET	/inventory	Returns current inventory ledger for the restaurant
GET	/inventory/export	Returns Excel file of current inventory
POST	/events	Creates an event with recipes and serving count
GET	/events/:id/reorder	Returns reorder list and shortage breakdown for the event
POST	/normalize	Internal endpoint called by the job worker after extraction

Async Job Flow

When a user uploads an invoice, the API stores the file in S3, creates an Invoice record with status pending, enqueues a BullMQ job, and returns the invoice ID. The job worker picks up the job, calls the document extraction service, then calls POST /normalize for each extracted line item. On

completion, the invoice status updates to done and a WebSocket push is sent to the client. On failure after three retries, status is set to error.

Ingredient Normalization Service

The normalization service resolves raw vendor-supplied ingredient strings to a single canonical ingredient ID. It runs a three-stage pipeline: preprocessing, alias exact match, and hybrid scoring.

Stage 1: Preprocessing

- 8. Lowercase and strip whitespace.
- 9. Remove quantity tokens such as "25LB", "3 1/2 c.", and "12oz" to isolate the ingredient name.
- 10. Expand abbreviations. "EVOO" becomes "extra virgin olive oil". "CHKN" becomes "chicken".
- 11. Remove invoice stopwords: "case", "bag", "box", "fresh", "organic".
- 12. Apply Porter stemming to reduce inflectional variation.

Example: "TOMATOES ROMA 25LB CASE" becomes "tomato roma" after preprocessing.

Stage 2: Alias Exact Match

The preprocessed string is looked up in the IngredientAlias table. If a match exists, confidence is set to 1.0 and the pipeline returns immediately. Vendor-scoped aliases are checked before global aliases.

Stage 3: Hybrid Scoring

If no alias match exists, each catalog candidate is scored using three weighted signals.

Signal	Method	Weight
Semantic similarity	Cosine similarity between sentence embeddings (all-MiniLM-L6-v2, 384 dimensions)	0.60
String similarity	Jaro-Winkler on preprocessed query vs canonical name	0.25
Token overlap	Jaccard coefficient over unigram token sets	0.15

A pgvector ivfflat index retrieves the top 50 nearest neighbors by cosine similarity before hybrid scoring runs. This avoids scoring the full catalog on every request.

Confidence Routing

Confidence Range	Decision	Action
0.88 and above	Auto commit	Write NormalizationMatch with is_confirmed true. Update InventoryLedger.

0.55 to 0.87	Needs review	Write NormalizationMatch with is_confirmed false. Show top 3 candidates in review UI.
Below 0.55	New ingredient	Create inactive IngredientCanonical placeholder. Notify admin.

Catalog Bootstrap

The canonical ingredient catalog is seeded from the RecipeNLG dataset (1.6 million recipes). The bootstrap process extracts ingredient names, clusters similar names using embedding cosine distance, selects the most frequent name in each cluster as the canonical, and stores all other names as aliases. The top 500 canonicals are manually reviewed before production use.

Configuration / External Interfaces

Environment Variables

Variable	Service	Source	Description
DATABASE_URL	Both	Secrets Manager	PostgreSQL connection string
REDIS_URL	Both	Secrets Manager	Redis connection string for BullMQ and cache
JWT_SECRET	Main API	Secrets Manager	Secret used to sign and verify JWT tokens
AWS_S3_BUCKET	Main API	ECS task env	S3 bucket name for invoice image storage
NORM_ALPHA	ML Service	ECS task env	Embedding weight in hybrid score (default 0.60)
NORM_BETA	ML Service	ECS task env	Jaro-Winkler weight in hybrid score (default 0.25)
NORM_GAMMA	ML Service	ECS task env	Token overlap weight in hybrid score (default 0.15)
HIGH_CONF_THRESHOLD	ML Service	ECS task env	Auto-commit threshold (default 0.88)
LOW_CONF_THRESHOLD	ML Service	ECS task env	Needs-review lower bound (default 0.55)

External Dependencies

- AWS S3: stores all uploaded invoice images and PDF files. The main API uses the AWS SDK to generate pre-signed upload URLs and to retrieve files for processing.
- AWS Secrets Manager: all credentials and API keys are fetched at container startup via the ECS secrets injection mechanism.
- pgvector PostgreSQL extension: must be enabled on the RDS instance before the ML service starts. The bootstrap script runs CREATE EXTENSION IF NOT EXISTS vector on first deploy.

- all-MiniLM-L6-v2 model: downloaded from Hugging Face on ML service container startup and cached in the container filesystem. Internet access is required on first cold start.
- BullMQ / Redis: the main API and ML service both connect to the same Redis instance. The job queue name is invoice-processing.

API Payload Examples

POST /invoices/upload

Request is multipart/form-data with the invoice file attached.

```
// Response
{
  "invoice_id": "a3f2c1d0-...",
  "status": "pending",
  "created_at": "2026-04-09T10:22:00Z"
}
```

POST /normalize (internal, called by job worker)

```
// Request
{
  "line_item_id": "7b91e4a2-...",
  "raw_text": "TOMATOES ROMA 25LB CASE",
  "raw_unit": "lb",
  "raw_qty": 25.0,
  "vendor_id": "c12d3e4f-..."
}

// Response (auto_commit example)
{
  "line_item_id": "7b91e4a2-...",
  "routing": "auto_commit",
  "top_match": {
    "canonical_id": "d9a3b2c1-...",
    "canonical_name": "Tomato",
    "confidence": 0.93,
    "method": "hybrid",
    "normalized_qty": 11.34,
    "normalized_unit": "kg"
  },
  "candidates": [
    { "canonical_name": "Tomato", "confidence": 0.93 },
    { "canonical_name": "Roma Tomato", "confidence": 0.81 },
    { "canonical_name": "Cherry Tomato", "confidence": 0.62 }
  ]
}
```

POST /invoices/:id/confirm

```
// Request
{
  "line_items": [
```

```
{
  "line_item_id": "7b91e4a2-...",
  "canonical_id": "d9a3b2c1-...",
  "confirmed_qty": 11.34,
  "confirmed_unit": "kg"
},
"user_id": "f1e2d3c4-..."
}

// Response
{
  "invoice_id": "a3f2c1d0-...",
  "status": "confirmed",
  "updated_items": 1
}
```

POST /events

```
// Request
{
  "name": "Saturday Catering",
  "date": "2026-04-19",
  "servings": 80,
  "recipes": [
    { "recipe_id": "r1a2b3c4-...", "servings": 80 },
    { "recipe_id": "r5d6e7f8-...", "servings": 80 }
  ]
}

// Response
{
  "event_id": "e9f0a1b2-...",
  "name": "Saturday Catering",
  "date": "2026-04-19",
  "ingredient_requirements": [
    { "canonical_name": "Tomato", "required_qty": 4.0, "unit": "kg" },
    { "canonical_name": "Chicken Breast", "required_qty": 12.0, "unit": "kg" }
  ]
}
```

Debug

Logging

All services emit structured JSON logs. Every log line includes a timestamp, log level, service name, and request ID. The request ID is set by the main API on each incoming request and passed to the ML service in the X-Request-ID header so all log lines for a single user action can be traced across services.

Key log events:

- Invoice upload received: logs invoice_id, file size, and mime type
- Job enqueued: logs job_id, invoice_id, and queue name
- Extraction complete: logs invoice_id, line item count, and processing duration
- Normalization result: logs line_item_id, raw_text, top confidence, and routing decision
- Job failed: logs job_id, attempt number, error message, and next retry delay
- User confirmation: logs match_id, original canonical_id, confirmed canonical_id, and user_id

Counters

The following metrics are emitted to AWS CloudWatch. Alarms are configured on the first two.

Metric Name	Description	Alarm Threshold
api.error_rate	Percentage of API requests returning 5xx in a 5-minute window	Alert if above 5%
api.latency_p95	p95 response time in milliseconds across all endpoints	Alert if above 2000ms
norm.auto_commit_rate	Percentage of normalization results routed to auto_commit	Log warning if below 70%
norm.override_rate	Percentage of auto-committed matches later corrected by users	Log warning if above 15%
queue.depth	Number of jobs waiting in the invoice-processing queue	Informational
queue.failure_rate	Percentage of jobs that exhausted all retries	Informational

Implementation

- The backend is organized into three separately deployable units: the main API, the ML service, and the shared PostgreSQL and Redis instances. Each unit has its own Dockerfile and can be updated independently.
- The main API is the only service that is publicly reachable. The ML service is internal and only called by the job worker inside the main API process. This keeps the normalization endpoint off the public internet.
- The job worker runs inside the same Node.js process as the main API. On each job, it fetches the invoice image from S3, calls the document extraction service for line items, then calls the ML service POST /normalize for each item. Results are written to the database and the invoice status is updated.
- The ML service loads the all-MiniLM-L6-v2 sentence transformer model once at startup. Canonical ingredient embeddings are pre-computed and stored in the ingredient_canonical table using the pgvector VECTOR(384) column type. At query time, only the input string is embedded at runtime.
- Database migrations are managed with a migration tool (node-postgres-migrate). Migrations run automatically as part of the CI/CD staging deployment step before the new containers start.

Cloud Deployment

Both services are built as Docker images in the CI/CD pipeline and pushed to AWS ECR tagged with the Git commit SHA. ECS task definitions reference the ECR image URI. Deployments update the task definition to the new image tag, ECS performs a rolling update, and the old tasks are drained before being terminated.

Component	AWS Service	Configuration
Main API	ECS Fargate	2 tasks minimum, scales to 6 on CPU above 70%
ML Service	ECS Fargate	1 task minimum, scales to 4 when queue depth exceeds 20
Database	RDS PostgreSQL 16	Multi-AZ, db.t4g.medium, automated 7-day backups
Redis	ElastiCache Redis 7	Single node staging, cluster mode production
Load Balancer	ALB	HTTPS on port 443, HTTP redirects to HTTPS, SSL via ACM
Container Registry	ECR	Images tagged by Git SHA, lifecycle rule keeps last 10

CI/CD Pipeline (GitHub Actions)

Stage	Trigger	Steps
Test	Every pull request	Unit tests, integration tests, and lint for both services
Build	Merge to main	Build Docker images, push to ECR with commit SHA tag
Deploy Staging	After build	Update ECS task definitions, run smoke tests
Deploy Production	Manual approval	Promote staging image tag to production ECS

Testing

General Approach

Tests are organized into three layers. Unit tests cover individual functions and run without a live database or network. Integration tests run against a test PostgreSQL instance seeded with 50 canonical ingredients and 200 labeled aliases. End-to-end tests cover the full invoice upload and processing pipeline against a local Docker Compose environment.

All tests run in CI on every pull request. Integration and end-to-end tests are also run as part of the staging deployment smoke test before traffic is promoted.

The offline normalization evaluation uses a labeled pairwise alias set constructed from RecipeNLG containing 1,000 pairs (500 true matches and 500 non-matches) held out from catalog construction. This evaluation runs weekly in CI and logs results to CloudWatch.

Unit Tests

Test ID	Function Under Test	Input	Expected Result
UT-01	preprocess()	"TOMATOES ROMA 25LB CASE"	Returns "tomato roma"
UT-02	preprocess()	"EVOO 1L"	Returns "extra virgin olive oil"
UT-03	preprocess()	"3 1/2 c. chicken broth"	Returns "chicken broth"
UT-04	route_confidence()	confidence = 0.93	Returns routing auto_commit
UT-05	route_confidence()	confidence = 0.70	Returns routing needs_review
UT-06	route_confidence()	confidence = 0.40	Returns routing new_ingredient
UT-07	convert_unit()	25.0, "lb"	Returns 11.34, "kg"
UT-08	convert_unit()	3.0, "c."	Returns 0.711, "l"
UT-09	Auth middleware	Request with expired JWT token	Returns HTTP 401 Unauthorized
UT-10	Job retry logic	Extraction job throws on first attempt	Job retries and succeeds on second attempt

Test ID	Integration Scenario	Pass Condition
IT-01	POST /normalize with known alias in DB	Returns confidence 1.0 and routing auto_commit
IT-02	POST /normalize with OCR typo "letuce"	Top candidate is Lettuce with confidence above 0.88
IT-03	POST /normalize with completely unknown string	Routing is new_ingredient and inactive placeholder created
IT-04	POST /invoices/:id/confirm on a pending match	is_confirmed set to true and new IngredientAlias row written
IT-05	POST /invoices/upload full end-to-end pipeline	Invoice status reaches done and inventory updated within 5 seconds
IT-06	GET /inventory/export	Returns valid Excel file matching current database state
IT-07	POST /normalize same line_item_id twice (idempotency)	Second call returns same canonical_id without creating a duplicate row

Appendix

Key Database Tables

IngredientCanonical

```
CREATE TABLE ingredient_canonical (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  canonical_name TEXT NOT NULL UNIQUE,
  category    TEXT,
  default_unit TEXT,
  embedding   VECTOR(384),
```

```
        is_active      BOOLEAN NOT NULL DEFAULT TRUE,
        created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
    );
CREATE INDEX idx_canonical_embedding ON ingredient_canonical
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```

IngredientAlias

```
CREATE TABLE ingredient_alias (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    canonical_id      UUID NOT NULL REFERENCES ingredient_canonical(id),
    alias_text        TEXT NOT NULL,
    alias_norm        TEXT NOT NULL,
    vendor_context    TEXT,
    source            TEXT NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (alias_norm, vendor_context)
);
```

NormalizationMatch

```
CREATE TABLE normalization_match (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    line_item_id      UUID NOT NULL REFERENCES invoice_line_item(id),
    canonical_id      UUID REFERENCES ingredient_canonical(id),
    raw_text          TEXT NOT NULL,
    confidence         NUMERIC(4,3) NOT NULL,
    method            TEXT NOT NULL,
    decided_by        TEXT NOT NULL,
    is_confirmed       BOOLEAN NOT NULL DEFAULT FALSE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Performance and Accuracy Targets

Metric	Target
Normalization pairwise F1 on labeled eval set	0.85 or above
Human override rate on auto-committed matches	Below 15%
Single invoice end-to-end processing time (p95)	Below 5 seconds
Batch normalization for 50 line items (p95)	Below 3 seconds
Single-item normalization latency (p95)	Below 200ms
API uptime in staging	99% or above